HW#00: MLP function approximation [100 pts]

Scientific Machine Learning

---

1 Theoretical Foundations [25 pts]

1.1 The Universal Approximation Theorem (UAT)

a) In 2-3 lines, state the Universal Approximation Theorem for a single-hidden-layer feedforward neural network.

b) Briefly discuss the limitations of the UAT. What does it guarantee, and what does it *not* guarantee about network training, architecture, or efficiency?

1.2 Bernstein Polynomials and a ReLU Network

The Bernstein polynomial of degree n for f on $[0, 1]$ is

$$B_n f(x) = \sum_{k=0}^{n} f\left(\tfrac{k}{n}\right) \binom{n}{k} x^k (1-x)^{n-k}.$$

Take $f_1(x) = \sin(\pi x)$ and $f_2(x) = |x - \frac{1}{2}|$.

a) For each function, plot $B_n f$ against f for $n = 4, 16, 64, 256$ and report the largest gap $\max_x |f(x) - B_n f(x)|$ for each n.

b) Repeat with a one-hidden-layer ReLU network of width $4, 16, 64, 256$, trained on 200 evenly spaced samples of f.

c) Which function does each representation handle better, and why?

1.3 Derivatives of Sigmoid and ReLU

a) For $\sigma(z) = (1 + e^{-z})^{-1}$, derive $\sigma'(z)$ by hand and show that $\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$.

b) For $\mathrm{ReLU}(z) = \max(0, z)$, write the derivative for $z < 0$ and for $z > 0$. What happens at $z = 0$?

c) Plot σ, σ', ReLU, and ReLU$'$ on $[-5, 5]$. Where is σ' small, and what does that do to a gradient that passes through such a unit during backpropagation?

2 Computational Experiments: Function Approximation [25 pts]

2.1 Approximating a Discontinuous Function

This problem explores the challenge of approximating non-smooth functions. Consider the Heaviside step function on the interval $x \in [0, 1]$:

$$H(x) = \begin{cases} 0 & x < 0.5 \\ 1 & x \geq 0.5 \end{cases}$$

a) Using a single hidden layer MLP with 50 neurons and a 'tanh' activation, train the network to approximate $H(x)$.

b) Plot the learned function $\hat{u}(x)$ against the true function $H(x)$. Describe the behavior around the discontinuity at $x = 0.5$. Does it resemble the Gibbs phenomenon?

c) Experiment to improve the approximation by independently:

- Increasing the hidden layer width (e.g., to 200, then 500 neurons).
- Increasing the network depth (e.g., 3 hidden layers of 50 neurons each).
- Changing the activation function to ReLU.
- Try different loss functions

d) **Analysis:** Why does a 'tanh'-based network struggle with discontinuities? Which modification produced the best result and why? Relate your findings to the fact that compositions of 'tanh' are C^∞, while ReLU networks are piecewise linear.

3 Compositional Learning [25 pts]

This section explores how to solve complex problems by breaking them into simpler, modular components that can be learned separately and then combined.

3.1 Learning and Composing Multi-Scale Functions

Consider the function $h(x) = f_{\text{linear}}(x) + f_{\text{high}}(x)$ on the interval $x \in [0, 1]$, where:

- $f_{\text{linear}}(x) = 1 + 2x$ (a simple linear trend)
- $f_{\text{high}}(x) = 0.5 \cos(20\pi x)$ (a high-frequency component)

Part 1: Learning the High-Frequency Component

To understand the challenge, first focus solely on approximating $f_{\text{high}}(x)$.

a) Train MLPs of increasing depth (1, 2, 3, and 4 hidden layers) to learn $f_{\text{high}}(x)$.

b) Plot the results on a 2x2 grid to visually demonstrate how approximation quality improves with depth. Explain this result in the context of spectral bias.

Part 2: Composition by "Stitching" Networks

Now, we will tackle the full problem $h(x)$ using a "divide and conquer" strategy.

a) Modular Training:

- Train a simple, shallow network, N_{linear}, to learn $f_{\text{linear}}(x)$.
- Train a separate, deep network, N_{high}, to learn $f_{\text{high}}(x)$.

b) Manual Composition: The final stitched model is formed by adding the outputs of the two trained networks:

$$\hat{h}_{\text{stitched}}(x) = N_{\text{linear}}(x) + N_{\text{high}}(x)$$

Implement this composition in code.

c) Comparison & Analysis: Train a single, deep end-to-end network to learn $h(x)$ directly. Create a side-by-side plot comparing the result of your stitched model against the end-to-end model. For clarity, zoom in on the interval $x \in [0.2, 0.8]$ for one of your plots. Discuss the results. Which approach is more effective and why?

4 Regularization [25 pts]

4.1 L1 vs. L2 Penalties

Take 40 evenly spaced points x_i on $[0, 2\pi]$ with readings $y_i = \sin(x_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, 0.1^2)$. Fit $\hat{y}(x) = \sum_{j=1}^{100} w_j \phi_j(x)$ with Gaussian bumps $\phi_j(x) = \exp\bigl(-(x - c_j)^2/2\ell^2\bigr)$, centers c_j evenly spaced on $[0, 2\pi]$ and $\ell = 0.5$, by minimizing

$$\frac{1}{40}\sum_{i=1}^{40}\bigl(\hat{y}(x_i) - y_i\bigr)^2 + \lambda R(\mathbf{w}), \qquad R(\mathbf{w}) = \sum_j w_j^2 \text{ (L2)} \quad \text{or} \quad R(\mathbf{w}) = \sum_j |w_j| \text{ (L1)}.$$

Solve the L1 problem by soft thresholding or with a library solver.

a) Fit with no penalty. Plot the fit against $\sin(x)$ and the readings. What happens between the readings, and why?

b) Fit with L2 and with L1 for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$. Report the RMSE against $\sin(x)$ for each.

c) At $\lambda = 10^{-2}$, plot the 100 weights of the L2 and L1 fits. How many are exactly zero in each, and why?